



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA

Lenguaje y compiladores trimestre 2026-I
Sección 01

Ing. Félix Márquez
fmarquez@e.uneg.edu.ve

Tema 3 (Lenguajes y Gramáticas Formales)

- 3.1. Jerarquía de Chomsky.
- 3.2. Alfabetos y palabras.
- 3.3. Lenguajes formales.
- 3.4. Gramáticas formales.
- 3.5. Expresiones Regulares.

Evaluación:

Informe de investigación 5% - defensa 15% (20 ptos)

- Exposición Adicional grabada con la resolución del cuestionario (BONUS PROACTIVIDAD 2%)

1.- Introducción.

Se pudo indagar en el tema anterior desde la práctica, desde el ver como están escrito los lenguajes formales, el propósito, sus paradigmas y en particular las estructuras morfológicas (léxicas) y sintácticas. Además, con el planteamiento de un problema y el proponer un lenguaje para resolver problemas de ese entorno, da cuenta que debemos conocer el como procesar a detalle un lenguaje L en particular, por tanto se hace necesario conocer herramientas teóricas que sirven para el procesamiento de L.

En el tema anterior, abordamos el estudio de los lenguajes de programación desde una perspectiva práctica: analizamos cómo se escriben, identificamos sus paradigmas fundamentales y examinamos sus estructuras morfológicas (léxicas) y sintácticas. Sin embargo, para diseñar, construir o incluso utilizar un compilador o intérprete de manera robusta e interesante, no basta con saber como se ve un lenguaje; es imperativo comprender cómo se genera y cómo se reconoce. Este tema nos introduce a los fundamentos



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA

teóricos del procesamiento de lenguajes, proporcionando las herramientas formales necesarias para modelar, validar y traducar cualquier lenguaje L.

En el nivel más atómico, comenzamos con los conceptos de alfabetos y palabras, que constituyen los bloques de construcción básicos. Un alfabeto no es más que un conjunto finito de símbolos, y una palabra es una secuencia finita de esos símbolos. Al agrupar estas palabras bajo reglas específicas, definimos un lenguaje formal, entendido no como un idioma humano, sino como un conjunto matemático de cadenas válidas. Esta abstracción es crucial, pues nos permite tratar el código fuente no como texto arbitrario, sino como un objeto matemático con propiedades demostrables.

El motor que da vida a estos lenguajes son las gramáticas formales. Inspiradas en los trabajos pioneros del lingüista Noam Chomsky en la década de 1950, las gramáticas proporcionan un conjunto finito de reglas de producción que permiten generar todas las cadenas válidas de un lenguaje y, a la inversa, validar si una cadena dada pertenece a él. En el contexto de la compilación, la gramática es el esqueleto que define la sintaxis del lenguaje de programación, permitiendo al compilador analizar la estructura jerárquica del código fuente y detectar errores sintácticos de manera temprana y precisa.

No todas las gramáticas tienen el mismo poder expresivo ni la misma complejidad de procesamiento. Aquí es donde la Jerarquía de Chomsky se convierte en una brújula indispensable. Esta clasificación divide las gramáticas en cuatro tipos (Tipo 0 hasta Tipo 3), estableciendo una relación directa entre la complejidad de las reglas y la capacidad computacional necesaria para procesarlas. Por ejemplo, entenderemos por qué las expresiones regulares (Tipo 3) son suficientes y altamente eficientes para el análisis léxico (identificar tokens como palabras clave o números), pero son matemáticamente incapaces de manejar estructuras anidadas como paréntesis balanceados o bloques de código, tarea que requiere Gramáticas Libres de Contexto (Tipo 2) y analizadores sintácticos (parsers) más sofisticados.

La teoría no se queda en el papel; se materializa a través de las Expresiones Regulares y los Autómatas Finitos (determinísticos y no determinísticos). Estos conceptos son la piedra angular de la implementación de los analizadores léxicos. Un autómata no es más que un modelo matemático de computación con memoria limitada, que actúa como un motor de reconocimiento de patrones. Dominar la conversión entre una expresión regular, un autómata y su implementación en código es una habilidad fundamental para cualquier ingeniero en informática.



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA

Es vital comprender que el alcance de estos fundamentos trasciende la construcción de compiladores tradicionales. Las mismas técnicas se aplican en la validación de formatos de datos modernos (como JSON o XML), en el procesamiento de lenguajes naturales (NLP), en la búsqueda de patrones en grandes volúmenes de texto, e incluso en la bioinformática, donde las secuencias de ADN se modelan y analizan como lenguajes formales sobre un alfabeto de cuatro letras.

Como se planteó anteriormente, al enfrentar un problema del mundo real y proponer un lenguaje específico para resolverlo, nos damos cuenta de la necesidad crítica de procesar dicho lenguaje con total certeza. Una gramática mal diseñada, que presente ambigüedad o recursividad por la izquierda no controlada, puede llevar a un compilador a bucles infinitos, a generar árboles de sintaxis contradictorios o a rechazar código válido. Por ello, el proceso de higiene de las gramáticas no es un mero ejercicio académico, sino una práctica de ingeniería esencial.

El objetivo de esta unidad va más allá de la memorización de definiciones. Buscamos fomentar un pensamiento computacional riguroso: la capacidad de abstraer un problema, definir sus reglas de manera formal e inequívoca, y traducir ese modelo a un algoritmo o autómata que lo resuelva. A través de los retos planteados, se les invita a dejar de ser meros consumidores de software para convertirse en arquitectos capaces de manipular los cimientos de la informática moderna.

Es importante señalar que es relevante las fuentes o documentos donde se extrae la información, de ello dependerá la calidad de los resultados, bien podemos considerar libros, artículos de revistas científicas, trabajos para optar a títulos de pregrado o postgrado, etc. Esta acotación se realiza para que evalúen la calidad de la fuente (documento, página, trabajo) donde están obteniendo su información.

Considero importante al aprovechar las herramientas de IA en sus investigaciones o tareas, recuerden que, sin embargo ciertas condiciones aplican, lo mas importante, su rol como analistas de lenguaje implica un análisis crítico y riguroso. La IA puede generar respuestas convincentes, pero no garantiza precisión ni neutralidad. Por esto, es obligatorio: 1) Exigir fuentes: Siempre soliciten a la IA las referencias específicas (libros, artículos, repositorios de código) que sustentan su respuesta. 2) Verificar y contrastar: Cruzar esa información con fuentes académicas confiables (documentación oficial, papers revisados por pares, repositorios como GitHub o Stack Overflow). 3) Cuestionar el origen: La referencia existe?, es reciente? viene de un autor o comunidad reconocida en el ámbito técnico?.



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA

Lo que le planteo no es mero procedimiento, es la base del pensamiento computacional sólido. Tenga en cuenta que un bug en su código por confiar en datos no validados puede ser tan grave como un error conceptual en su diseño o en la programación.

Condición sinecuanon, se busca que el equipo de forma colaborativa se apropie del conocimiento, la gran ventaja todas los recursos disponibles para ellos validar soluciones y contrastar con las diferentes IA. De modo que Usted sera evaluado por cuanto de muestre que a aprendido del tema cuanto lo ha reflexionado evidenciado en su defensa.

2. Preguntas de investigación y desarrollo.

2.1.- Fundamentos y Jerarquía (Teoría Aplicada).

2.2.1.- Relación Gramática-Lenguaje: Indique el concepto y al menos un ejemplo práctico de la relación entre Gramáticas y Lenguajes Formales. Explique detalladamente el mecanismo mediante el cual una gramática genera un lenguaje.

2.2.2.- Jerarquía de Chomsky: Explique los 4 tipos de gramáticas de la Jerarquía de Chomsky. Para cada tipo, proporcione un ejemplo práctico representado en notación BNF (Backus-Naur Form).

2.2.- Derivación y Modelado (Caso Práctico: Genoma)

2.2.1.- Gramática del para dibujar: Basado en una Gramática Libre de Contexto (GLC) que modele herramientas de dibujo (alfabeto $\Sigma = \{a, c, g, t\}$), presente 5 ejemplos de derivación de sus producciones para casos propuestos. Ejemplo de las derivaciones necesarias para dibujar: a) cuadrado, b) arbol con varias ramas y hojas, c) cubo, etc.

2.3.- Higiene y Optimización de Gramáticas

2.3.1.- Patologías de las Gramáticas: Las gramáticas mal diseñadas causan errores en los compiladores. Ejemplifique con 3 casos prácticos diferentes: a) Una Gramática Ambigua (demuestre la ambigüedad con dos árboles de derivación distintos para la misma cadena). b) Un caso de Recursividad por la Izquierda y muestre el algoritmo paso a paso para eliminarla. c) Un caso que requiera Factorización por la Izquierda y muestre la gramática resultante optimizada.



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA

2.4) De la Expresión al Autómata (Caso Práctico: Ajedrez).

2.4.1.- Notación PGN (Portable Game Notation): Plantee y ejecute autómatas finitos para el reconocimiento de un subconjunto del lenguaje PGN (Ajedrez).

2.4.2.- Defina un subconjunto simplificado del PGN para movimientos básicos.

2.4.3. Diseñe la Expresión Regular (Regex) y el Autómata Finito Determinístico (AFD) equivalente.

3. Protocolo de Entrega.

El proceso de evaluación está diseñado en dos partes para ponderar equitativamente el trabajo colaborativo formal en investigación escrita y la solvencia argumentativa de carácter técnico-individual:

Fase I: Evaluación Grupal - Informe Escrito y Repositorio (Ponderación: 10 Puntos) El repositorio debe incluir de forma obligatoria:

- El informe de investigación estructurado bajo las secciones descritas, exportado en formato PDF de alta calidad tipográfica.
- Los directorios independientes que contengan los códigos fuentes completamente desarrollados y funcionales para los entornos de compilación de los lenguajes trabajados.
- En el repositorio se validará: El historial de confirmaciones (commits) para certificar la participación activa y equitativa de cada integrante del equipo a lo largo del ciclo de desarrollo.

Fase II: Evaluación Individual - Defensa Técnica Grabada (Ponderación: 10 Puntos) Cada integrante del equipo, de forma estrictamente individual, debe presentar y defender el proyecto mediante un recurso audiovisual. El protocolo para la defensa incluye las siguientes condiciones operativas ineludibles:

- Tiempo Límite por Participante: Máximo 10 minutos de duración por cada orador. la capacidad de síntesis técnica será ponderada explícitamente.
- Criterios de Calidad Audiovisual: El video debe ser grabado con máxima claridad técnica. Es obligatorio garantizar que el rostro del expositor permanezca visible en primer plano o en recuadro a lo largo de toda la disertación. El audio debe poseer una nitidez acústica, libre de ruidos de fondo disruptivos o distorsiones que dificulten la correcta inteligibilidad de los juicios expuestos.



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA

- Contenido Esencial de la Exposición: El participante debe justificar conceptualmente una de las actividades asignadas, explicar el funcionamiento lógico-sintáctico del código escrito y responder a las implicaciones del compilador frente al lenguaje seleccionado, demostrando apropiación de la teoría de lenguajes y compiladores.

4. Guía de Evaluación.

Se presenta tabla con el resumen de los componente a evaluar con su respectiva ponderación.

Componente Evaluado	Criterios de Excelencia Técnica	Ponderación
Calidad del Informe Escrito y APA	Cumplimiento exhaustivo de la estructura formal, coherencia en la matriz paradigmática y uso de las referencia normas APA.	1 Ptos
Ingeniería Algorítmica y Repositorio	Cocrrta implementación de los algoritmos. y uso del repositorio git.	1 Ptos
Innovación, orden precisión en los planteamientos y ejemplos presentados	Orden y rigurosidad teórica en la presentación de los planteamientos y ejemplos	8.0 Ptos
Dominio de Conceptos en Defensa (Individual)	Fluidez discursiva, precisión técnica rigurosa al explicar cómo interactúan las fases del compilador/intérprete con su código e independencia analítica respecto al uso de herramientas de IA.	6.0 Ptos
Sintetización	Cumplimiento exacto del límite estricto de 10 minutos por	4.0 Ptos



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA

Audiovisual Restricciones (individual)	y participante. Calidad óptima de audio, perfecta visibilidad del rostro expuesto y encuadre técnico adecuado.	
PUNTUACIÓN MÁXIMA TOTAL ACUMULADA		20.0 Ptos